Makiti Mojapelo

Exercise 3: SQL CASE Statements

1. Classify each product by price tier (table: products)

Steps

1. Select **product_name** and **price** from products.
2. Add a CASE: test the biggest tier first (price > 1000), then BETWEEN 100 AND 1000, ELSE covers the rest (< 100).
3. END the CASE and alias it **AS price_category**.

Query

```
SELECT product_name, price,
    CASE
        WHEN price > 1000 THEN 'Expensive'
        WHEN price BETWEEN 100 AND 1000 THEN 'Mid-range'
        ELSE 'Budget'
    END AS price_category
FROM products;
```

Working

- Laptop: 1200 > 1000 → Expensive
- Phone: 800 is between 100 and 1000 → Mid-range
- Keyboard: 45 → Budget
- Monitor: 300 → Mid-range
- Mouse: 25 → Budget

Output

product_name	price	price_category
Laptop	1200.00	Expensive
Phone	800.00	Mid-range
Keyboard	45.00	Budget
Monitor	300.00	Mid-range
Mouse	25.00	Budget

2. Label each order by its value (table: orders)

Steps

1. Select **customer_name** and **amount** from orders.
2. Check >= 1000 first, then 500 to 999.99, ELSE is Low (< 500).
3. Alias the CASE **AS order_value_category**.

Query

```
SELECT customer_name, amount,
  CASE
    WHEN amount >= 1000 THEN 'High Value'
    WHEN amount BETWEEN 500 AND 999.99 THEN 'Medium Value'
    ELSE 'Low Value'
  END AS order_value_category
FROM orders;
```

Working

- Alice: 150 < 500 → Low Value
- Bob: 560 is in 500–999.99 → Medium Value
- Charlie: 999.99 is in range → Medium Value
- Diana: 45.50 → Low Value
- Ethan: 1200 >= 1000 → High Value

Output

customer_name	amount	order_value_category
Alice	150.00	Low Value
Bob	560.00	Medium Value
Charlie	999.99	Medium Value
Diana	45.50	Low Value
Ethan	1200.00	High Value

3. Categorize each employee's position level (table: employees)

Steps

1. Select **emp_name, department, salary.**
2. Both department AND salary must match, so join the tests with AND.
3. Anything that fails both rules falls to ELSE 'Staff'. Alias **AS position_level.**

Query

```
SELECT emp_name, department, salary,
  CASE
    WHEN department = 'IT' AND salary > 80000 THEN 'Senior IT'
    WHEN department = 'HR' AND salary > 55000 THEN 'Experienced HR'
    ELSE 'Staff'
  END AS position_level
FROM employees;
```

Working

- John: IT and 85000 > 80000 → Senior IT
- Sara: HR and 60000 > 55000 → Experienced HR
- Mark: IT but 75000 is not > 80000 → Staff
- Lucy: Finance matches no rule → Staff
- Tom: HR but 55000 is not > 55000 → Staff

Output

emp_name	department	salary	position_level
John	IT	85000	Senior IT
Sara	HR	60000	Experienced HR
Mark	IT	75000	Staff
Lucy	Finance	95000	Staff
Tom	HR	55000	Staff

4. Assign each student a letter grade (table: students)

Steps

1. Select **student_name** and **score**.
2. Work from the top grade down: >= 90, >= 80, >= 70, >= 60, ELSE F.
   (CASE stops at the first true WHEN, so no upper limits are needed.)
3. Alias **AS grade**.

Query

```
SELECT student_name, score,
  CASE
    WHEN score >= 90 THEN 'A'
    WHEN score >= 80 THEN 'B'
    WHEN score >= 70 THEN 'C'
    WHEN score >= 60 THEN 'D'
    ELSE 'F'
  END AS grade
FROM students;
```

Working

- Anna: 92 → A
- Ben: 76 → C
- Cara: 59 → F
- David: 83 → B
- Ella: 68 → D

Output

student_name	score	grade
Anna	92	A
Ben	76	C
Cara	59	F
David	83	B
Ella	68	D

5. Label delivery performance (table: deliveries)

Steps

1. Select **delivery_id** and **delivery_time_minutes**.
2. <= 30 is Fast, <= 60 is On Time (31-60 because Fast was already caught), ELSE Late (> 60).
3. Alias **AS performance**.

Query

```
SELECT delivery_id, delivery_time_minutes,
  CASE
    WHEN delivery_time_minutes <= 30 THEN 'Fast'
    WHEN delivery_time_minutes <= 60 THEN 'On Time'
    ELSE 'Late'
  END AS performance
FROM deliveries;
```

Working

- 1: 45 → On Time
- 2: 80 > 60 → Late
- 3: 30 <= 30 → Fast
- 4: 65 > 60 → Late
- 5: 100 → Late

Output

delivery_id	delivery_time_minutes	performance
1	45	On Time
2	80	Late
3	30	Fast
4	65	Late
5	100	Late

6. Convert numeric priority into a label (table: tickets)

Steps

1. Select **issue_type** and **priority**.
2. Exact matches only, so use a simple CASE: CASE priority WHEN 3 ...
3. Alias **AS priority_label**.

Query

```
SELECT issue_type, priority,
  CASE priority
    WHEN 3 THEN 'High'
    WHEN 2 THEN 'Medium'
    WHEN 1 THEN 'Low'
  END AS priority_label
FROM tickets;
```

Working

- Login issue: 1 → Low
- Server down: 3 → High
- Slow system: 2 → Medium
- Email error: 2 → Medium
- Password reset: 1 → Low

Output

issue_type	priority	priority_label
Login issue	1	Low
Server down	3	High
Slow system	2	Medium
Email error	2	Medium
Password reset	1	Low

7. Calculate attendance % and classify it (table: attendance)

Steps

1. Percentage = days_present x 100.0 / total_days. (Use 100.0 so SQL does decimal, not integer, division.)
2. Select **student_id**, the percentage **AS attendance_percentage**.
3. CASE on the same calculation: >= 90 Excellent, >= 75 Good, ELSE Needs Improvement. Alias **AS attendance_status**.

Query

```
SELECT student_id,
    days_present * 100.0 / total_days AS attendance_percentage,
    CASE
        WHEN days_present * 100.0 / total_days >= 90 THEN 'Excellent'
        WHEN days_present * 100.0 / total_days >= 75 THEN 'Good'
        ELSE 'Needs Improvement'
    END AS attendance_status
FROM attendance;
```

Working

- 1: 45 x 100 / 50 = 90 → Excellent (90 counts, it is >= 90)
- 2: 30 x 100 / 50 = 60 → Needs Improvement
- 3: 48 x 100 / 50 = 96 → Excellent
- 4: 25 x 100 / 50 = 50 → Needs Improvement
- 5: 50 x 100 / 50 = 100 → Excellent

Output

student_id	attendance_percentage	attendance_status
1	90	Excellent
2	60	Needs Improvement
3	96	Excellent
4	50	Needs Improvement
5	100	Excellent

8. Label the stock status (table: products_inventory)

Steps

1. Select **product_id** and **stock_qty**.
2. 0 is Out of Stock, BETWEEN 1 AND 5 is Low Stock, ELSE (> 5) In Stock.
3. Alias **AS stock_status**.

Query

```
SELECT product_id, stock_qty,
  CASE
    WHEN stock_qty = 0 THEN 'Out of Stock'
    WHEN stock_qty BETWEEN 1 AND 5 THEN 'Low Stock'
    ELSE 'In Stock'
  END AS stock_status
FROM products_inventory;
```

Working

- 1: 5 → Low Stock
- 2: 0 → Out of Stock
- 3: 25 → In Stock
- 4: 10 → In Stock
- 5: 3 → Low Stock

Output

product_id	stock_qty	stock_status
1	5	Low Stock
2	0	Out of Stock
3	25	In Stock
4	10	In Stock
5	3	Low Stock

9. Classify each class by size (table: classes)

Steps

1. Select **subject** and **enrolled_students**.
2. >= 25 Large, BETWEEN 10 AND 24 Medium, ELSE (< 10) Small.
3. Alias **AS class_size_category**.

Query

```
SELECT subject, enrolled_students,
  CASE
    WHEN enrolled_students >= 25 THEN 'Large'
    WHEN enrolled_students BETWEEN 10 AND 24 THEN 'Medium'
    ELSE 'Small'
  END AS class_size_category
FROM classes;
```

Working

- Math: 30 → Large
- English: 25 >= 25 → Large
- Science: 15 → Medium
- Art: 5 → Small
- History: 20 → Medium

Output

subject	enrolled_students	class_size_category
Math	30	Large
English	25	Large
Science	15	Medium
Art	5	Small
History	20	Medium

10. Apply a discount flag (table: payments)

Steps

1. Select **payment_id, payment_method, amount.**
2. Both conditions must be true, so join them with AND: method = 'Cash' AND amount >= 200.
3. Everything else is ELSE 'Not Eligible'. Alias **AS discount_eligibility.**

Query

```
SELECT payment_id, payment_method, amount,
  CASE
    WHEN payment_method = 'Cash' AND amount >= 200
      THEN 'Eligible for Discount'
    ELSE 'Not Eligible'
  END AS discount_eligibility
FROM payments;
```

Working

- 1: Card → Not Eligible
- 2: Cash and 200 >= 200 → Eligible for Discount
- 3: Card → Not Eligible
- 4: PayPal → Not Eligible
- 5: Cash and 300 >= 200 → Eligible for Discount

Output

payment_id	payment_method	amount	discount_eligibility
1	Card	50.00	Not Eligible
2	Cash	200.00	Eligible for Discount
3	Card	150.00	Not Eligible
4	PayPal	75.00	Not Eligible
5	Cash	300.00	Eligible for Discount

Rule used throughout: CASE checks each WHEN from top to bottom and stops at the first one that is true.